Homework #2

Vanja Stojanović

April 2026

1 Where is SN100?

We study the given bottlenose dolphine network. Given that the disappearance of the dolphine SN100, yields distinct communities, it is natural to explore its centrality measures. In particular the degree, betweenness, closeness, and eigenvector centralities. With these we can detected how important SN100 was structurally, i.e. seeing if it formed bridges between communities. Results of the measures can be seen in Table 1 below.

Table 1: Top 5 dolphins by each centrality measure. SN100 is **bolded** where it appears.

	Betweenness			*Closeness*	
Rank	Dolphin	Value	Rank	Dolphin	Value
1	**SN100**	0.248	1	**SN100**	0.418
2	Beescratch	0.213	2	SN9	0.404
3	SN9	0.143	3	SN4	0.399
4	SN4	0.139	4	Kringel	0.391
5	DN63	0.118	5	Grin	0.377
	Degree			*Eigenvector*	
Rank	Dolphin	Value	Rank	Dolphin	Value
1	Grin	0.197	1	Grin	0.316
2	SN4	0.180	2	SN4	0.301
3	Topless	0.180	3	Topless	0.285
4	Scabs	0.164	4	Scabs	0.281
5	Trigger	0.164	5	TR99	0.218

SN100 ranks 1st in both betweenness and closeness centrality despite only ranking 17th in degree ($k = 7$). This confirms its role as broker, it bridges otherwise

distant sub-communities rather than being a highly connected hub. Its low eigenvector centrality (rank 19) further indicates that its importance comes from its structural position, not from being connected to other hubs. We can further confim this by looking at the neighbourhood of SN100: Beescratch, Kringel, MN60, SN4, SN89, SN9, Zap. These neighbours also appear high in the measures given in the Table 1 above, in particular SN4 is rank 2 in degree, and eigenvector centrality.

The following is the Python implementation used to calculate the centrality measures.

```
import networkx as nx
import sys


def parse_pajek(path):
    G = nx.Graph()
    labels = {}
    with open(path) as f:
        lines = [l.strip() for l in f if l.strip()]
    i = 0
    # skip to *vertices
    while i < len(lines) and not lines[i].lower().startswith("*vertices"):
        i += 1
    n = int(lines[i].split()[1])
    i += 1
    for _ in range(n):
        parts = lines[i].split()
        vid = int(parts[0])
        name = parts[1].strip('"')
        G.add_node(vid, label=name)
        labels[name] = vid
        i += 1
    # skip to *edges
    while i < len(lines) and not lines[i].lower().startswith("*edges"):
        i += 1
    i += 1
    while i < len(lines) and not lines[i].startswith("*"):
        parts = lines[i].split()
        G.add_edge(int(parts[0]), int(parts[1]))
        i += 1
    return G, labels


G, labels = parse_pajek("networks/dolphins.net")
target = "SN100"
tid = labels[target]

```

```python
print(f"Network: {G.number_of_nodes()} nodes, {G.number_of_edges()} edges\n")

# Degree centrality
dc = nx.degree_centrality(G)
dc_sorted = sorted(dc.items(), key=lambda x: -x[1])
rank_dc = next(i for i, (v, _) in enumerate(dc_sorted, 1) if v == tid)

# Betweenness centrality
bc = nx.betweenness_centrality(G)
bc_sorted = sorted(bc.items(), key=lambda x: -x[1])
rank_bc = next(i for i, (v, _) in enumerate(bc_sorted, 1) if v == tid)

# Closeness centrality
cc = nx.closeness_centrality(G)
cc_sorted = sorted(cc.items(), key=lambda x: -x[1])
rank_cc = next(i for i, (v, _) in enumerate(cc_sorted, 1) if v == tid)

# Eigenvector centrality
ec = nx.eigenvector_centrality(G, max_iter=1000)
ec_sorted = sorted(ec.items(), key=lambda x: -x[1])
rank_ec = next(i for i, (v, _) in enumerate(ec_sorted, 1) if v == tid)

print(f"=== {target} (node {tid}) ===")
print(
    f"  Degree:       {dc[tid]:.4f}  (deg={G.degree(tid)})  rank {rank_dc}/{G.number_of_nodes()}"
)
print(f"  Betweenness:  {bc[tid]:.4f}  rank {rank_bc}/{G.number_of_nodes()}")
print(f"  Closeness:    {cc[tid]:.4f}  rank {rank_cc}/{G.number_of_nodes()}")
print(f"  Eigenvector:  {ec[tid]:.4f}  rank {rank_ec}/{G.number_of_nodes()}")

print(f"\nNeighbors of {target}: {[G.nodes[v]['label'] for v in G.neighbors(tid)]}")

print("\n=== Top 10 by each centrality ===")
for name, ranked in [
    ("Degree", dc_sorted),
    ("Betweenness", bc_sorted),
    ("Closeness", cc_sorted),
    ("Eigenvector", ec_sorted),
]:
```

```
    print(f"\n{name}:")
    for v, val in ranked[:10]:
        marker = "␣<--" if v == tid else ""
        print(f"␣␣{G.nodes[v]['label']:15s}␣␣{val:.4f}{marker}")
```

2 HIV and network sampling

We approximate contact tracing via a simple random walk on the social network, stopping once 15% of nodes have been visited. The induced subgraph on the visited nodes forms our sampled network. Table 2 summarizes the structural properties of both networks.

Table 2: Structural properties of the original and sampled networks.

Property	Original	Sampled (15% RW)
Nodes	10680	1602
Edges	24316	8247
Average degree $\langle k \rangle$	4.55	10.30
Clustering C	0.2659	0.4580
C_{rand}	0.0004	0.0064
C/C_{rand}	623.69	71.21
Avg. shortest path L (GCC)	7.49	5.16
L_{rand}	6.12	3.16
L/L_{rand}	1.22	1.63
Power-law $\hat{\gamma}$	1.91	1.59
Power-law R^2	0.93	0.82

Original network. The clustering coefficient is $\sim$624$\times$ higher than expected for a random graph, while the average path length is only $\sim$1.2$\times$ the random baseline. This is characteristic of a small-world network. The degree distribution fits a power law with $\hat{\gamma} \approx 1.91$ and $R^2 = 0.93$, suggesting the network is also scale-free.

Sampled network. The sampled network remains small-world ($C \gg C_{\text{rand}}$, $L \approx L_{\text{rand}}$). However, the power-law fit degrades ($R^2 = 0.82$, $\hat{\gamma} \approx 1.59$). The random walk introduces a degree bias: high-degree nodes are visited disproportionately often (the stationary distribution of a random walk is proportional to node degree). This inflates the average degree from 4.55 to 10.30 and compresses the degree distribution, weakening the scale-free signature. The elevated clustering ($C = 0.46$ vs. 0.27) is also a consequence of this bias, the walk tends to stay within densely connected clusters.

The following is the Python implementation used.

```
import networkx as nx
from networkx import Graph
import random
import numpy as np
from collections import Counter
```

```python

random.seed(42)


def parse_pajek(path):
    G = nx.Graph()
    labels = {}
    with open(path) as f:
        lines = [l.strip() for l in f if l.strip()]
    i = 0
    while i < len(lines) and not lines[i].lower().startswith("*vertices"):
        i += 1
    n = int(lines[i].split()[1])
    i += 1
    for _ in range(n):
        parts = lines[i].split()
        vid = int(parts[0])
        name = parts[1].strip('"')
        G.add_node(vid, label=name)
        labels[name] = vid
        i += 1
    while i < len(lines) and not lines[i].lower().startswith("*edges"):
        i += 1
    i += 1
    while i < len(lines) and not lines[i].startswith("*"):
        parts = lines[i].split()
        G.add_edge(int(parts[0]), int(parts[1]))
        i += 1
    return G, labels


def random_walk(G: Graph, threshold: float):
    start_node = random.choice(list(G.nodes))
    rw = [start_node]
    visited = {start_node}

    while len(visited) < int(threshold * G.number_of_nodes()):
        next_node = random.choice(list(G.neighbors(rw[-1])))
        rw.append(next_node)
        visited.add(next_node)

    return rw


def analyze_network(G, name):
    print(f"\n=== {name} ===")
    print(f"Nodes: {G.number_of_nodes()}, Edges: {G.
```

```
        number_of_edges()}")

    avg_deg = 2 * G.number_of_edges() / G.number_of_nodes()
    print(f"Average degree: {avg_deg:.2f}")

    C = nx.average_clustering(G)
    print(f"Clustering coefficient: {C:.4f}")

    # Compare to random graph
    n = G.number_of_nodes()
    m = G.number_of_edges()
    p = 2 * m / (n * (n - 1))
    C_rand = p
    print(f"Expected C for random graph: {C_rand:.4f}")
    print(f"C / C_rand: {C / C_rand:.2f}")

    # Average shortest path length on largest connected component
    gcc = max(nx.connected_components(G), key=len)
    G_gcc = G.subgraph(gcc).copy()
    L = nx.average_shortest_path_length(G_gcc)
    L_rand = np.log(len(gcc)) / np.log(avg_deg) if avg_deg > 1
        else float('inf')
    print(f"Avg shortest path (GCC, n={len(gcc)}): {L:.4f}")
    print(f"Expected L for random graph: {L_rand:.4f}")
    print(f"L / L_rand: {L / L_rand:.2f}")

    small_world = C > 10 * C_rand and L / L_rand < 2
    print(f"Small-world: C >> C_rand and L ~ L_rand => {
        small_world}")

    # Degree distribution for scale-free check
    degrees = [d for _, d in G.degree()]
    deg_count = Counter(degrees)
    ks = sorted(deg_count.keys())
    # Log-log linear regression on CCDF
    ccdf = []
    total = len(degrees)
    for k in ks:
        ccdf.append(sum(1 for d in degrees if d >= k) / total)
    log_k = np.log(np.array(ks, dtype=float))
    log_ccdf = np.log(np.array(ccdf))
    coeffs = np.polyfit(log_k, log_ccdf, 1)
    gamma_est = -coeffs[0]
    residuals = log_ccdf - np.polyval(coeffs, log_k)
    ss_res = np.sum(residuals**2)
    ss_tot = np.sum((log_ccdf - np.mean(log_ccdf))**2)
    r_squared = 1 - ss_res / ss_tot
    print(f"Power-law fit: gamma ~ {gamma_est:.2f}, R^2 = {
        r_squared:.4f}")
```

```
    print(f"Max degree: {max(degrees)}, Min degree: {min(degrees)}")

    return {
        "n": n, "m": m, "avg_deg": avg_deg,
        "C": C, "C_rand": C_rand,
        "L": L, "L_rand": L_rand,
        "gamma": gamma_est, "r2": r_squared,
    }


G, labels = parse_pajek("networks/social.net")
rw = random_walk(G, 0.15)
visited = set(rw)
G_sampled = G.subgraph(visited).copy()

orig = analyze_network(G, "Original social network")
samp = analyze_network(G_sampled, "Sampled network (15% RW)")
```

3 Ring graph modularity

Homework #2

The homework does not require a lot of writing but may require some thinking. It does not require a lot of processing power but may require efficient programming. It accounts for 12.5% of the course grade. Any questions or comments should be posted on Piazza.

Submission details

This homework is due on **May 1st** at 11:59pm. It must be submitted through (1) Gradescope (entry code **RVYW5D**) and (2) eUcilnica. (1) Submission to Gradescope should include answers to all questions, each on a separate page, which may also demand pseudocode, proofs, tables, plots, diagrams and/or other. (2) Submission to eUcilnica should include this cover sheet with signed honor code and all the programming code used to complete the exercises (preferably in `.py` format). The homework is considered submitted only when *both* (1) and (2) have been submitted. Failing to include the honor code in the submission will result in **10% deduction**. Failing to submit all the developed code will result in **10% deduction**.

Honor code

Students are strongly encouraged to discuss the homework with other classmates and form study groups. However, each student must then solve the homework by herself/himself without the help of others and should be able to redo the homework at a later time. In other words, students are encouraged to collaborate but should not copy from one another. Referring to any solutions obtained from classmates, course books, previous years, found online, AI tools or other is considered an honor code violation. Also, stating any part of the solutions in class or on Piazza is considered an honor code violation.

Any violation of the honor code will be reported to the **faculty disciplinary committee** and vice dean for education.

SID: _____

Full name: _____

Study group: _____

I understand and accept the honor code.

Signature: _____